MÔN HỌC EMBEDDED LINUX SYSTEMS

BÁO CÁO THỰC HÀNH

LAB-02

VIẾT DEVICE DRIVER & QUẢN LÝ FILE SYSTEM

Kernel Module • Character Device • procfs/sysfs • MTD/JFFS2 • BusyBox Init

THÔNG TIN	NỘI DUNG
Sinh viên thực hiện	Lê Tiến Thành
Mã số sinh viên	SE201799
Nhóm thực hành	Lê Tiến Thành; Lê Tấn Lộc; Nguyễn Quang Huy Đạt; Trần Trương Tuấn Vũ
Nền tảng đích	ARMv7-A Cortex-A9, QEMU vexpress-a9
Kernel / hệ thống	Linux 5.15.0 / BusyBox init / initramfs
Ngày tổng hợp báo cáo	20/10/2026

Báo cáo dựa trên log và ảnh chụp thực nghiệm do sinh viên cung cấp.

TÓM TẮT KẾT QUẢ THỰC HÀNH

Báo cáo mô tả việc biên dịch và kiểm thử character device driver lab2_driver trên Linux 5.15 dành cho ARMv7; xuất dữ liệu qua procfs/sysfs; mô phỏng NAND trên Ubuntu Host và vận hành JFFS2; tích hợp nạp module tự động bằng BusyBox init trong QEMU.

Kết quả chính: module ARM ELF32 có vermagic tương thích 5.15.0; /dev/lab2 được đăng ký với major 240/minor 0; các thao tác read/write và hai script kiểm thử đều báo PASS; nandsim nhận NAND 16 MiB, eraseblock 16 KiB; JFFS2 được ghi vào MTD, mount đọc được, và dữ liệu vẫn tồn tại sau unmount/mount lại. Các kết luận được giới hạn trong phạm vi ảnh/log đã ghi nhận.

NHÓM YÊU CẦU	TÌNH TRẠNG	BẰNG CHỨNG CHÍNH
A – Kernel Module / Character Device	ĐẠT	ARM ELF32; vermagic; major 240; read/write; test_driver.sh PASS
B – procfs / sysfs	ĐẠT	/proc/lab2_info và ba attribute; test_procfs.sh PASS
C – MTD / JFFS2	ĐẠT	mtdinfo, flash_erase, nandwrite, mount, remount, nanddump
D – BusyBox Init / QEMU	ĐẠT	rcS auto-load; /dev/lab2; hai test script; initramfs archive

NỘI DUNG BÁO CÁO

- 1. Thông tin sinh viên và môi trường
- 2. Sơ đồ kiến trúc driver
- 3. Log và kết quả thực hành (A, B, C, D)
- 4. So sánh filesystem
- 5. Trả lời 5 câu hỏi phân tích
- 6. Nhận xét, khó khăn và kết luận
- Phụ lục – Lệnh kiểm chứng và tài liệu tham khảo

Quy ước: “HOST” là Ubuntu x86_64 của máy phát triển; “TARGET” là Linux ARMv7-A trong QEMU. Ảnh chụp terminal được lấy từ quá trình thực hành ngày 20/10/2026.

1. THÔNG TIN SINH VIÊN VÀ MÔI TRƯỜNG

1.1. Mục tiêu

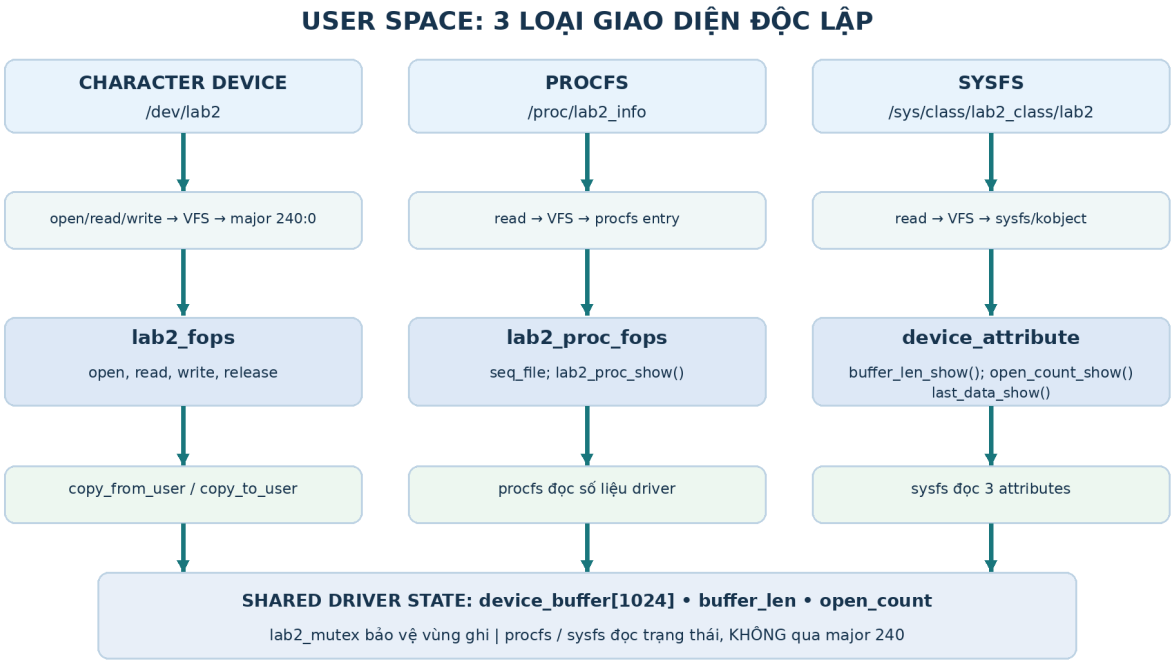
Thực hành theo LAB-02: (A) viết và biên dịch character device driver; (B) công bố thông tin qua procfs/sysfs; (C) thao tác NAND giả lập qua MTD và JFFS2; (D) đưa kernel module vào initramfs và nạp tự động khi BusyBox init khởi động.

1.2. Thiết lập thí nghiệm

HẠNG MỤC	CẤU HÌNH / GIÁ TRỊ QUAN SÁT
HOST	Ubuntu Linux, CPU x86_64; Kernel 6.8.0-147-generic (uname -r)
TARGET	ARMv7-A Cortex-A9, QEMU -M vexpress-a9 -cpu cortex-a9 -m 512M -smp 2
Kernel target	Linux 5.15.0, SMP, module unloading
Cross toolchain	arm-linux-gnueabihf-; GCC 13.3.0 theo log biên dịch kernel LAB-01
Module	lab2_driver.ko, ELF32 relocatable, ARM EABI5; version 1.0; GPL
Device node	/dev/lab2 (major 240, minor 0)
procfs	/proc/lab2_info
sysfs	/sys/class/lab2_class/lab2/
NAND (HOST)	nandsim, /dev/mtd0, 16 MiB, eraseblock 16 KiB, page 512 B, OOB 16 B
Filesystem	JFFS2, read-write trên /mnt/jffs2

Lưu ý về khác biệt so với ví dụ trong tài liệu: lệnh nandsim thực tế sử dụng second_id_byte=0x33, kết quả tạo NAND 16 MiB. Đề bài có ví dụ chip 32 MB dùng mã nhận dạng khác; báo cáo ghi đúng thông số do mtdinfo thực tế cung cấp. Thử nghiệm JFFS2 diễn ra trên HOST, còn driver character và BusyBox boot diễn ra trên TARGET.

2. SƠ ĐỒ KIẾN TRÚC DRIVER (BẮT BUỘC)



Hình 1. Kiến trúc tổng quát: userspace → system calls → VFS → file_operations → device_buffer; procfs/sysfs là giao diện trạng thái.

2.1. Đường đi của lời gọi hệ thống

Ứng dụng trong userspace truy cập character device qua /dev/lab2. VFS dựa vào major 240 và file_operations để chuyển các lời gọi open(), read(), write(), release() tới lab2_open(), lab2_read(), lab2_write(), lab2_release(). Bộ đệm nội bộ device_buffer có kích thước định nghĩa 1024 byte.

2.2. Luồng dữ liệu khi ghi / đọc

```
WRITE: user buffer → write() → VFS → lab2_write() → copy_from_user() → device_buffer
      → /proc/lab2_info (thống kê) &
/sys/class/lab2_class/lab2/{buffer_len,open_count,last_data}
READ : device_buffer → lab2_read() → copy_to_user() → user buffer
```

Trạng thái dữ liệu được công bố dưới dạng bản thống kê tại /proc/lab2_info và từng thuộc tính riêng tại /sys/class/lab2_class/lab2/. Trong bài lab, ba thuộc tính buffer_len, open_count, last_data được khai báo read-only. Mutex có nhiệm vụ giảm xung đột khi nhiều tiến trình cùng thay đổi dữ liệu.

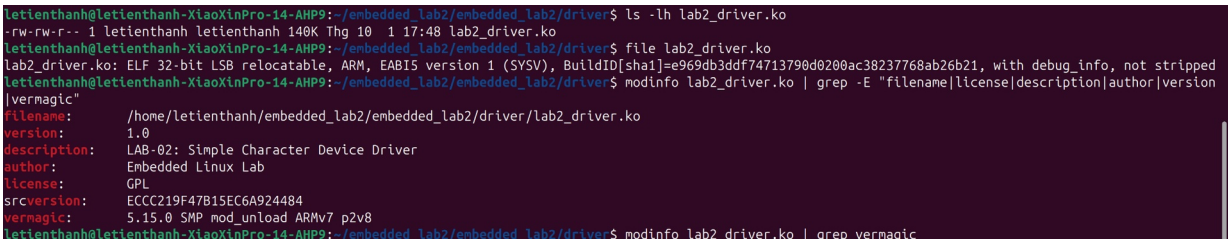
2.3. Tổ chức sysfs

```
/sys/class/lab2_class/lab2/
├── buffer_len       (RO, độ dài dữ liệu cuối)
├── open_count       (RO, số lần mở thiết bị)
└── last_data        (RO, dữ liệu gần nhất)
```

3. LOG VÀ KẾT QUẢ THỰC HÀNH (PHẦN A – D)

3.1. Phần A – Biên dịch và kiểm tra định dạng

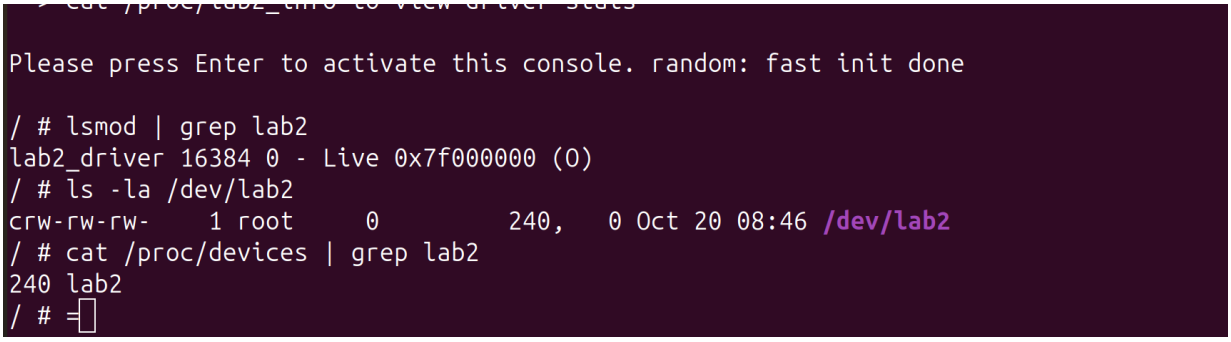
Hệ thống build KBuild tạo lab2_driver.ko cho ARM. Modinfo ghi nhận license GPL, MODULE_VERSION 1.0 và vermagic "5.15.0 SMP mod_unload ARMv7 p2v8". Đây là bước kiểm tra tương thích trước khi insmod module vào kernel target.



Hình 2. Kết quả file/modinfo của lab2_driver.ko: kiến trúc ARM EABI5 và vermagic kernel 5.15.0.

3.1.1. Đăng ký character device

Khi module được nạp, log cho biết "lab2_driver: loaded, major=240"; lsmod có lab2_driver và node /dev/lab2 là character device với major/minor 240,0, chế độ crw-rw-rw-. /proc/devices đồng thời có dòng "240 lab2". Các quan sát xác thực rằng đường dẫn thiết bị ánh xạ tới driver đã đăng ký.



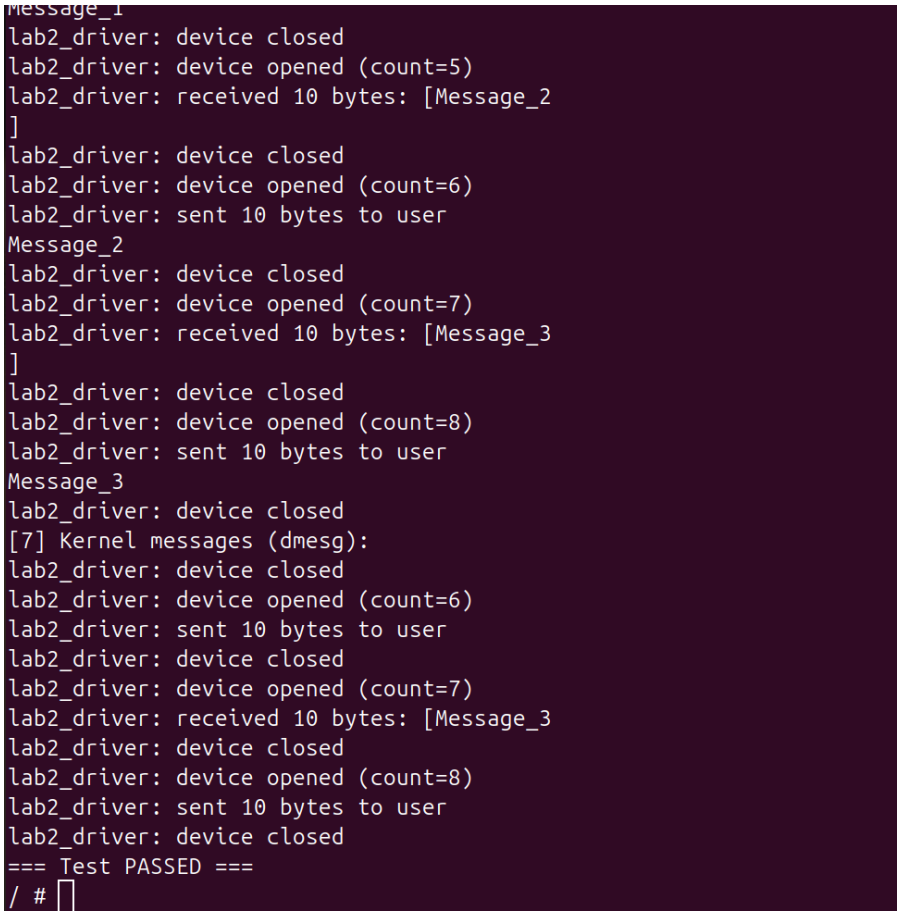
Hình 3. Trên TARGET: lsmod, /dev/lab2 và /proc/devices cùng xác nhận major 240.

3.1.2. Thao tác đọc / ghi và test script

Script test_driver.sh đã được chạy thành công. Ghi "Hello from userspace, LAB-02!" vào /dev/lab2 và đọc lại cho chuỗi trùng khớp; vòng lặp ghi/đọc Message_1, Message_2, Message_3 cũng cho kết quả đúng. Driver đồng thời in kernel log về số byte và vòng đời open/release.

```
/ # test_driver.sh
=== LAB-02: Device Driver Test ===
[1] Checking lab2_driver...
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
 (node already exists)
crw-------    1 root     0         240,   0 Oct 20 08:51 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
 Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1
]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2
]
lab2_driver: device closed
```

Hình 4. test_driver.sh: kiểm tra module, node, WRITE/READ và phần đầu bài test nhiều lượt.



Hình 5. test_driver.sh (tiếp): các chu kỳ Message_2/Message_3 và dòng "Test PASSED".

3.1.3. Ghi nhận kernel log

Log dmesg có các sự kiện device opened, received 10/30 bytes, sent bytes to user, device closed, module unloaded và initializing/loaded sau khi nạp lại. Điều này cho phép kiểm tra các callback file_operations, không chỉ dựa vào kết quả từ cat.

```
=== procfs/sysfs Test PASSED ===
/ # echo "===== LAB2 FINAL KERNEL LOG ====="
===== LAB2 FINAL KERNEL LOG =====
/ # dmesg | grep lab2 | tail -30
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: module unloaded
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
lab2_driver: device closed
/ #
```

Hình 6. Một phần dmesg thể hiện open/read/write/release, unload và load lại module.

3.2. PHẦN B – GIAO TIẾP QUA PROCFS VÀ SYSFS

3.2.1. Giao diện procfs

Entry /proc/lab2_info dùng procfs/seq_file để hiển thị thống kê. Ở trạng thái mới nạp module trong test_procfs.sh, Data length = 0 và Open count = 0. Sau khi ghi “Embedded Linux Lab2 Test Data”, Data length chuyển thành 30 bytes, Open count thành 1 và Last data chứa đúng nội dung đã ghi.

```
/ # echo "===== RUN TEST_PROCFS.SH ====="
===== RUN TEST_PROCFS.SH =====
/ # test_procfs.sh
=== Test procfs & sysfs ===
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 0 bytes
Open count : 0
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 1
Last data : [Embedded Linux Lab2 Test Data
]
```

Hình 7. test_procfs.sh: phần log procfs trước/sau WRITE; kết quả sysfs và Test PASSED xem ở Hình 6 và Hình 8.

3.2.2. Giao diện sysfs

Theo kiểm thử thực tế, các file buffer_len, open_count và last_data được đọc tại /sys/class/lab2_class/lab2/. Giá trị trên ảnh là buffer_len = 30, open_count = 18 và last_data = “Embedded Linux Lab2 Test Data” (số đếm thay đổi theo lần truy cập). SYSFS biểu diễn từng thuộc tính của một thiết bị, khác với báo cáo nhiều trường trong một file procfs.

```
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
======================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 18
Last data : [Embedded Linux Lab2 Test Data
]
/ # echo "===== SYSFS ATTRIBUTES ====="
===== SYSFS ATTRIBUTES =====
/ #
/ # SYSFS=/sys/class/lab2_class/lab2
/ #
/ # ls -la $SYSFS
lrwxrwxrwx    1 root     0               0 Oct 20 08:51 /sys/class/lab2_class/lab2 -> ../../devices/virtual/lab2_class/lab2
/ #
/ # echo "buffer_len = $(cat $SYSFS/buffer_len)"
buffer_len = 30
/ # echo "open_count = $(cat $SYSFS/open_count)"
open_count = 18
/ # echo "last_data  = $(cat $SYSFS/last_data)"
last_data  = Embedded Linux Lab2 Test Data
```

Hình 8. Đọc ba attribute buffer_len, open_count, last_data từ sysfs (bằng chứng riêng).

3.2.3. Đánh giá

Hai hình thức cung cấp thông tin đều hoạt động. /proc thuận lợi cho một bản thống kê nhiều dòng; /sys thuận lợi cho từng trường trạng thái của một device kobject. Theo code mẫu của tài liệu, ba sysfs attributes là read-only (DEVICE_ATTR_RO), không phải thuộc tính cho phép ghi.

3.3. PHẦN C – MTD SUBSYSTEM VÀ JFFS2 FILE SYSTEM

3.3.1. Khởi tạo NAND giả lập trên HOST

Thực nghiệm dùng nandsim với first_id_byte=0x20 và second_id_byte=0x33. /proc/mtd xác nhận "mtd0: 01000000 00004000 NAND simulator partition 0". Chuyển đổi từ hex cho kết quả 16 MiB dung lượng và 16 KiB mỗi eraseblock.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0x33
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /proc/mtd
ls -l /dev/mtd*
dev:    size   erasesize  name
mtd0: 01000000 00004000 "NAND simulator partition 0"
crw------- 1 root root 90, 0 Thg 10 20 15:58 /dev/mtd0
crw------- 1 root root 90, 1 Thg 10 20 15:58 /dev/mtd0ro

/dev/mtd:
total 0
drwxr-xr-x 2 root root 60 Thg 10 20 15:58 by-name
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 9. nandsim tạo mtd0 và hai node /dev/mtd0, /dev/mtd0ro trên Ubuntu HOST.

3.3.2. Thông số NAND và erase

mtdinfo ghi NAND, 1024 eraseblocks × 16 KiB, minimum I/O 512 B, OOB 16 B, bad blocks allowed và writable = true. Lệnh sudo flash_erase /dev/mtd0 0 0 hoàn thành 100%, chứng minh thiết bị raw flash giả lập đã được xóa trước khi ghi image.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mtdinfo /dev/mtd0
mtd0
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                16384 bytes, 16.0 KiB
Amount of eraseblocks:          1024 (16777216 bytes, 16.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size:                  256 bytes
OOB size:                       16 bytes
Character device major/minor:   90:0
Bad blocks are allowed:         true
Device is writable:             true

letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo flash_erase /dev/mtd0 0 0
Erasing 16384 Kibyte @ 0 -- 100 % complete
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 10. mtdinfo và flash_erase 100% trên NAND giả lập 16 MiB.

3.3.3. Tạo image JFFS2 và ghi NAND

Bộ dữ liệu mẫu được tổ chức dưới jffs2_root/etc/lab2.txt và jffs2_root/data/test.txt. Lệnh mkfs.jffs2 -d jffs2_root -o rootfs.jffs2 -e 0x4000 -n tạo image được công cụ file nhận diện là "Linux jffs2 filesystem data little endian". Tiếp theo nandwrite -p /dev/mtd0 rootfs.jffs2 thông báo ghi block 0 tại offset 0x0.

```
rootfs.jffs2: Linux jffs2 filesystem data little endian
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cd ~/embedded_lab2/embedded_lab2

mkdir -p jffs2_root/etc
printf "Embedded Linux Lab 2 - JFFS2\n" > jffs2_root/etc/lab2.txt

find jffs2_root -type f -o -type d

rm -f rootfs.jffs2

mkfs.jffs2 -d jffs2_root -o rootfs.jffs2 -e 0x4000 -n

ls -lh rootfs.jffs2
file rootfs.jffs2
jffs2_root
jffs2_root/data
jffs2_root/data/test.txt
jffs2_root/etc
jffs2_root/etc/lab2.txt
-rw-r--r-- 1 letienthanh letienthanh 512 Thg 10 20 16:01 rootfs.jffs2
rootfs.jffs2: Linux jffs2 filesystem data little endian
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo nandwrite -p /dev/mtd0 rootfs.jffs2
Writing data to block 0 at offset 0x0
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 11. Tạo rootfs.jffs2 (512 B) và nandwrite thành công vào NAND giả lập.

3.3.4. Mount và đọc dữ liệu

Sau khi nạp mtdblock, /dev/mtdblock0 xuất hiện. Lệnh mount -t jffs2 /dev/mtdblock0 /mnt/jffs2 chạy thành công. File etc/lab2.txt chứa “Embedded Linux Lab 2 - JFFS2”, data/test.txt chứa “Persistent data test”. Output mount xác nhận filesystem jffs2 (rw,relatime).

```
Try: sudo apt install <deb name>
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo modprobe mtdblock
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ ls -l /dev/mtdblock*
brw-rw---- 1 root disk 31, 0 Thg 10 20 16:02 /dev/mtdblock0
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ find /mnt/jffs2 -type f -o -type d
/mnt/jffs2
/mnt/jffs2/etc
/mnt/jffs2/etc/lab2.txt
/mnt/jffs2/data
/mnt/jffs2/data/test.txt
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/etc/lab2.txt
Embedded Linux Lab 2 - JFFS2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 12. JFFS2 mount ở chế độ read-write và đọc đúng hai file đã lưu.

3.3.5. Kiểm chứng persistence sau remount

Bài test ghi nối thêm “Persistent after remount” vào data/test.txt, tạo /mnt/jffs2/runtime.txt chứa “Created during runtime”, gọi sync, unmount và mount lại. Sau remount, cả hai nội dung vẫn đọc được, không bị mất. Đây là bằng chứng duy trì dữ liệu qua chu kỳ tháo/gắn lại filesystem trong khi thiết bị NAND giả lập vẫn tồn tại.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Persistent after remount\n" >> /mnt/jffs2/data/test.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Created during runtime\n" > /mnt/jffs2/runtime.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sync
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo umount /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 13. Trước và sau umount/mount: dữ liệu được ghi trong JFFS2 vẫn còn nguyên.

Giới hạn: nandsim lưu NAND trong RAM. Kết quả chứng minh persistence qua remount, không phải qua mất điện hoặc reboot HOST; không nên nhầm với độ bền dữ liệu trên NAND vật lý.

3.3.6. Dump dữ liệu raw NAND

nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd0 ghi 512 byte từ offset 0; báo ECC failed = 0, ECC corrected = 0 và bad blocks = 0. hexdump cho thấy hai byte đầu 85 19: magic JFFS2 ở dạng little-endian (giá trị 0x1985). Tùy chọn -o bỏ phần OOB khỏi dump, không phải thêm OOB.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd0
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ hexdump -C /tmp/dump.bin | head -20
00000000  85 19 01 e0 2c 00 00 00  5f 56 f1 e0 01 00 00 00  |....,...._V......|
00000010  00 00 00 00 02 00 00 00  93 2d d7 6a 04 00 00 00  |.........-.j....|
00000020  67 32 eb d2 7f 2c b7 8c  64 61 74 61 85 19 02 e0  |g2...,..data....|
00000030  44 00 00 00 1d fb f7 98  02 00 00 00 01 00 00 00  |D...............|
00000040  fd 41 00 00 e8 03 e8 03  00 00 00 00 93 2d d7 6a  |.A...........-.j|
00000050  93 2d d7 6a 93 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 47 7c 5a d2  |............G|Z.|
00000070  85 19 01 e0 2b 00 00 00  e6 6e 26 7d 01 00 00 00  |....+....n&}....|
00000080  01 00 00 00 03 00 00 00  db 2d d7 6a 03 04 00 00  |.........-.j....|
00000090  78 0e 89 80 db 85 f4 d1  65 74 63 ff 85 19 02 e0  |x.......etc.....|
000000a0  44 00 00 00 1d fb f7 98  03 00 00 00 01 00 00 00  |D...............|
000000b0  fd 41 00 00 e8 03 e8 03  00 00 00 00 db 2d d7 6a  |.A...........-.j|
000000c0  db 2d d7 6a db 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
000000d0  00 00 00 00 00 00 00 00  00 00 00 00 9c 8f c9 d3  |................|
000000e0  85 19 01 e0 30 00 00 00  78 be 3e fa 02 00 00 00  |....0...x.>.....|
000000f0  02 00 00 00 04 00 00 00  93 2d d7 6a 08 08 00 00  |.........-.j....|
00000100  f2 b9 13 56 12 9b eb d3  74 65 73 74 2e 74 78 74  |...V....test.txt|
00000110  85 19 02 e0 59 00 00 00  5f 74 84 3a 04 00 00 00  |....Y..._t.:....|
00000120  01 00 00 00 b4 81 00 00  e8 03 e8 03 15 00 00 00  |................|
00000130  93 2d d7 6a 93 2d d7 6a  93 2d d7 6a 00 00 00 00  |.-.j.-.j.-.j....|
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 14. nanddump và hexdump: JFFS2 magic bytes 85 19, ECC failed 0, bad blocks 0.

3.3.7. Cleanup nandsim và xác nhận Kernel Host

Sau khi kiểm chứng dữ liệu, unmount /mnt/jffs2, gỡ các module mtdblock và nandsim trên Ubuntu Host. Kết quả kiểm tra cho thấy “MTD device removed”; uname -r xác nhận Kernel Host 6.8.0-147-generic. Đây là thao tác cleanup phía HOST, không thực hiện bên trong QEMU.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo umount /mnt/jffs2
sudo modprobe -r mtdblock
sudo modprobe -r nandsim
ls /dev/mtd0 2>/dev/null || printf 'MTD device removed\n'
uname -r
[sudo] password for letienthanh:
MTD device removed
6.8.0-147-generic
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ ls /dev/mtd0 2>/dev/null || printf 'MTD device removed\n'
uname -r
MTD device removed
6.8.0-147-generic
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

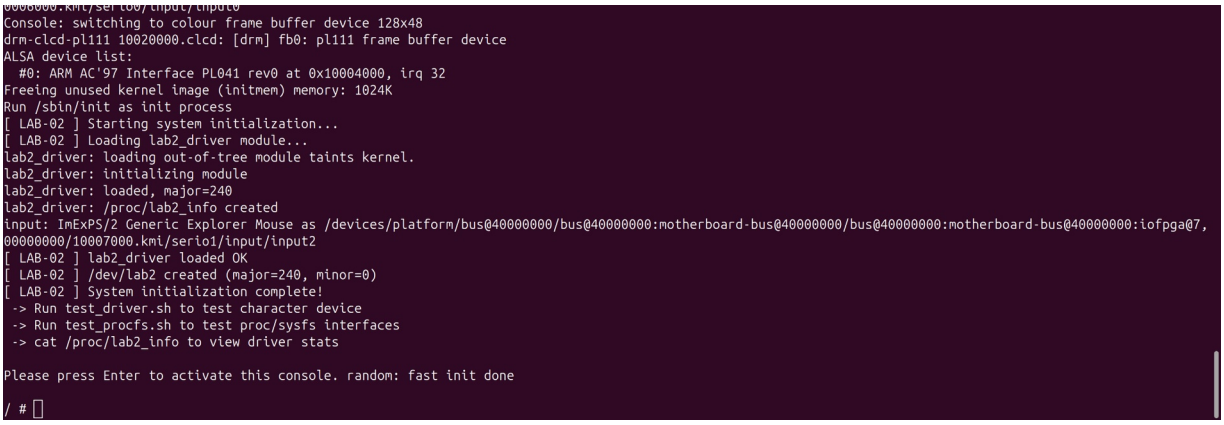
Hình 15. HOST: umount, modprobe -r mtdblock/nandsim và xác nhận /dev/mtd0 đã biến mất; uname -r = 6.8.0-147-generic.

3.4. PHẦN D – BUSYBOX INIT VÀ AUTO-LOAD MODULE

3.4.1. Chuỗi khởi động

```
Kernel 5.15.0 → /sbin/init (BusyBox) → /etc/inittab
     → ::sysinit:/etc/init.d/rcS
     → mount proc/sys/dev → insmod lab2_driver.ko
     → /dev/lab2 (240:0) → BusyBox shell
```

Theo yêu cầu Lab 2, rcS thực hiện các thao tác chuẩn bị môi trường và insmod module; nếu load thành công, tạo node /dev/lab2. Ảnh boot thực tế ghi “[LAB-02] lab2_driver loaded OK”, “[LAB-02] /dev/lab2 created (major=240, minor=0)” và “System initialization complete!”.



Hình 16. Boot QEMU: BusyBox rcS nạp lab2_driver tự động và tạo character node.

3.4.2. Kiểm tra gói initramfs

Archive initramfs_lab2.cpio.gz được kiểm tra bằng zcat kết hợp cpio -t. Kết quả cho thấy ba file cần thiết bên trong: module /lib/modules/5.15.0/lab2_driver.ko và hai script /usr/bin/test_driver.sh, /usr/bin/test_procfs.sh.



Hình 17. Danh sách file trong initramfs_lab2.cpio.gz: module và hai script kiểm thử.

3.4.3. Kiểm thử sau boot và điều kiện thành công

Trong BusyBox shell: lsmod nhận diện lab2_driver; /dev/lab2 mang major 240; test_driver.sh kết thúc “Test PASSED”; test_procfs.sh kết thúc “procfs/sysfs Test PASSED”; các giá trị /proc và /sys khớp với dữ liệu ghi. Module cũng có thể gỡ/nạp lại theo dmesg. Những bằng chứng này đã được trình bày ở Phần A/B, tránh chèn trùng ảnh gây dài báo cáo.

4. SO SÁNH FILESYSTEM VÀ LỰA CHỌN CHO EMBEDDED

4.1. Bảng so sánh theo ít nhất 5 tiêu chí

FILESYSTEM	GHI ?	WEAR LEVELING	NÉN	KHỞI ĐỘNG / MOUNT	THIẾT BỊ PHÙ HỢP
ext4	Có	Không trực tiếp; dựa FTL phần cứng	Không mặc định	Nhanh trên block device; journal recovery	eMMC / SD / SSD / HDD
JFFS2	Có	Có (log + garbage collection)	Có	Có thể chậm do quét flash khi mount	Raw NOR / raw NAND dung lượng nhỏ
cramfs	Không	Không	Có	Thường nhanh, dữ liệu tĩnh	Firmware/ROM chỉ đọc
UBIFS	Có	Qua UBI (wear-leveling)	Có	Thường tốt hơn JFFS2 trên NAND lớn	Raw NAND, cần UBI

SquashFS	Không	Không	Có, tỷ lệ cao	Đọc/mount hiệu quả; phụ thuộc tầng lưu trữ	Rootfs firmware bất biến

Nhận xét: ext4 được thiết kế cho block device nên không ghi trực tiếp lên raw MTD; JFFS2 làm việc với các eraseblock flash; UBIFS dùng lớp UBI để quản lý volume, bad blocks và wear leveling. cramfs/SquashFS thích hợp image chỉ đọc hơn là dữ liệu runtime cần thay đổi.

4.2. Đề xuất cho IoT có 32 MB NAND flash và OTA

Với NAND raw cỡ 32 MB, có thể chọn JFFS2 cho phân vùng dữ liệu cấu hình/log nhỏ và cân nhắc một rootfs firmware chỉ đọc (ví dụ SquashFS đi kèm một tầng lưu trữ raw-flash phù hợp, không mount trực tiếp như ext4). Nếu cấu hình và dung lượng cho phép, thiết kế firmware A/B, xác thực image, rollback và cơ chế cập nhật an toàn để tránh mất khả năng boot khi OTA bị gián đoạn. Nếu hệ thống mở rộng lên NAND lớn hơn và tải ghi nhiều, UBIFS+UBI thường đáng cân nhắc hơn JFFS2.

Bài thực nghiệm chạy NAND 16 MiB là minh họa kỹ thuật MTD/JFFS2; không phải đánh giá thực tế hiệu năng hay độ tin cậy của sản phẩm OTA 32 MB.

5. TRẢ LỜI CÂU HỎI PHÂN TÍCH (MỤC VI)

Câu 1. Vì sao dùng copy_to_user()/copy_from_user() thay vì memcpy()?

ARMv7 32-bit sử dụng không gian địa chỉ ảo 4 GiB. Trong cấu hình CONFIG_VMSPLIT_2G đã chọn tại LAB-01, ranh giới điển hình là user 0x00000000–0x7FFFFFFF và kernel 0x80000000–0xFFFFFFFF (khác mặc định VMSPLIT_3G thường gặp). Phần user thuộc mỗi tiến trình và được ánh xạ/bảo vệ bởi MMU; phần kernel được quản lý bằng các ánh xạ và quyền truy cập của nhân. Đây là mô tả phân chia địa chỉ ảo, không đồng nghĩa user có quyền đọc mọi địa chỉ trong dải của nó.

copy_from_user()/copy_to_user() là API uaccess: kiểm tra dải địa chỉ user hợp lệ (cơ chế access_ok tùy kiến trúc/đường gọi), rồi truy cập an toàn với hỗ trợ page fault phù hợp. Một trang hợp lệ có thể cần fault-in qua demand paging; con trỏ không hợp lệ có thể gây Data Abort. Kernel dùng các mục fixup trong exception table (__ex_table) ở đường truy cập uaccess để khôi phục có kiểm soát và trả lại số byte không copy thành công; driver có thể chuyển lỗi này thành -EFAULT. memcpy() không tự mang hợp đồng kiểm tra/fixup đó khi nhận user pointer; truy cập sai có thể gây kernel Oops, lỗi bộ nhớ hoặc rủi ro bảo mật tùy ánh xạ và cấu hình, không nên khẳng định mọi memcpy đều lập tức thành khai thác thành công.

copy_from_user() lấy dữ liệu từ địa chỉ userspace vào device_buffer trong lab2_write(); copy_to_user() trả dữ liệu từ kernel buffer về userspace trong lab2_read(). Hai hàm được xây dựng theo quy ước uaccess và trả số byte chưa copy thành công, từ đó driver có thể trả -EFAULT khi có lỗi. Quan sát thực nghiệm: ghi “Hello from userspace, LAB-02!” qua /dev/lab2 rồi đọc lại nguyên vẹn, chứng minh đường truyền dữ liệu hoạt động.

Câu 2. Vai trò mutex và race condition khi có nhiều process

DEFINE_MUTEX(lab2_mutex) bảo vệ device_buffer và buffer_len trong lab2_write(). copy_from_user(device_buffer, buf, len) có thể sleep vì page fault, và QEMU -smp 2 cho phép hai CPU thực thi xen kẽ. Nếu bỏ mutex: P1 copy 100 byte, chưa gán buffer_len; P2 ghi 5 byte ở đầu, gán buffer_len=5 và device_buffer[5]=\0; rồi P1 gán buffer_len=100 và device_buffer[100]=\0. Kết quả báo chiều dài 100 nhưng chuỗi có thể kết thúc ở byte thứ 5 hoặc bị trộn nội dung (torn write). Lock khiến các phép copy/cập nhật trạng thái ghi là một critical section; lưu ý read hiện tại cũng cần được xem xét đồng bộ để có snapshot nhất quán.

Kết quả log test_driver.sh ghi/đọc Message_1, Message_2, Message_3 tuần tự đều đúng. Tuy vậy, đây chưa phải kiểm thử stress đồng thời; để bảo đảm nhất quán cả khi read/write cạnh tranh, implementation cũng nên đồng bộ đường đọc hoặc dùng cơ chế snapshot phù hợp, không chỉ khóa khi write.

Câu 3. procfs và sysfs khác nhau thế nào?

procfs ban đầu thiên về tiến trình và trạng thái hệ thống; theo thời gian, các file nhiều dòng và định dạng văn bản tự do khiến userspace khó parse ổn định. sysfs gắn thiết bị với kobject và device_attribute, theo nguyên tắc cơ bản “one value per file” – mỗi attribute là một giá trị/thuộc tính đơn giản có ngữ nghĩa xác định. Vì vậy các thuộc tính mới của thiết bị thường nên đặt tại sysfs (hoặc giao diện chuyên dụng khác nếu thích hợp), không mở rộng /proc tùy ý. Nguyên tắc này có ngoại lệ được tài liệu kernel xác định, không nên diễn giải thành mọi file sysfs tuyệt đối chỉ chứa đúng một token.

Trong Lab 2, /proc/lab2_info hiển thị tên driver, major, buffer size, data length, open count, last data; sysfs tách buffer_len, open_count, last_data thành các file read-only. Ảnh cho thấy cả hai phản ánh cùng dữ liệu sau khi ghi 30 byte.

Câu 4. Vì sao NAND/JFFS2 cần erase block?

NAND flash không hỗ trợ ghi đè tùy ý từ bit 0 trở lại 1; trạng thái đã lập trình thường chỉ được khôi phục bằng thao tác erase theo cả eraseblock, không phải mỗi sector nhỏ. Trong khi đó, ext4 hoạt động trên block device có tầng điều khiển/FTL xử lý chi tiết lưu trữ và remapping. Vì vậy, áp dụng trực tiếp mô hình ghi đè của ext4 lên raw NAND không phù hợp.

JFFS2 là log-structured: thay đổi được ghi thành node mới ở vùng flash trống thay vì sửa tại chỗ, node cũ thành obsolete; garbage collection thu hồi eraseblock khi thích hợp, đồng thời phân bổ chu kỳ erase để hạn chế hao mòn. Bằng chứng: HOST báo eraseblock 16384 B, flash_erase 100%, nandwrite thành công, mount JFFS2 rw và đọc file sau remount.

Câu 5. Nếu insmod trong rcS lỗi, boot có tiếp tục không?

BusyBox init gọi rcS theo action sysinit. Nếu một lệnh insmod thất bại nhưng rcS chỉ in cảnh báo và vẫn chạy tới cuối, quá trình init thông thường tiếp tục đến console, dù chức năng /dev/lab2 không hoạt động. Nếu chính rcS gọi exit 1, điều đó kết thúc script với mã lỗi, không mặc nhiên gây kernel panic hay tắt máy; cần chính sách rõ ràng để xác định đó là lỗi nghiêm trọng hay có thể bỏ qua.

Cách cải thiện: kiểm tra tồn tại module, kiểm tra mã thoát của insmod, tạo device node chỉ khi load thành công, ghi log cụ thể, tránh nạp lặp khi module đã tồn tại. Với thiết bị mà driver là thành phần thiết yếu, có thể chuyển sang shell phục hồi hoặc đường khởi động dự phòng thay vì âm thầm tiếp tục.

```
MODULE_PATH=/lib/modules/5.15.0/lab2_driver.ko
MODULE_CRITICAL=no  # yes nếu đây là module bắt buộc
if grep -q "^lab2_driver " /proc/modules; then
    printf "Already loaded\n"
elif [ -f "$MODULE_PATH" ] && insmod "$MODULE_PATH"; then
    [ -c /dev/lab2 ] || mknod /dev/lab2 c 240 0
    chmod 666 /dev/lab2
else
    if [ "$MODULE_CRITICAL" = yes ]; then
        printf "FATAL: lab2_driver failed\n" >&2
        /sbin/poweroff -f
        exit 1
    else
        printf "WARN: lab2_driver failed; continue boot\n" >&2
    fi
fi
```

Trong lần boot được chụp, nhánh thành công đã chạy (“lab2_driver loaded OK”). Đoạn cải thiện ở trên là đề xuất chưa kiểm thử. Trong môi trường sản phẩm, nhánh fatal cần xác định chính sách recovery cụ thể; poweroff -f là lựa chọn minh họa có chủ ý, không chỉ exit 1 khỏi rcS.

6. NHẬN XÉT, KHÓ KHĂN VÀ KẾT LUẬN

6.1. Khó khăn thực tế

TÌNH HUỐNG	NGUYÊN NHÂN	XỬ LÝ / BÀI HỌC
Ban đầu không có /proc/mtd, /dev/mtd0; mtdinfo báo không có MTD.	nandsim trên HOST chưa được nạp.	Nạp nandsim, kiểm tra cat /proc/mtd trước mọi lệnh erase/write.
Terminal báo “command not found” khi dán một số lệnh, có ký tự ^[[200~ hoặc dấu ~.	Escape sequence của bracketed paste bị dính vào chuỗi lệnh.	Chạy lại câu lệnh sạch, kiểm tra lỗi thực tế; không sử dụng ảnh chứa lỗi làm bằng chứng chính.
Lần đầu thư mục jffs2_root/etc chưa tồn tại, dẫn đến thất bại khi tạo lab2.txt.	Lệnh mkdir bị paste sai nên không chạy.	Tạo lại thư mục/file, kiểm tra find trước khi mkfs.jffs2; tạo lại image rồi nandwrite.
Ví dụ NAND 32 MB trong đề khác output 16 MiB.	Mã chip giả lập chọn second_id_byte=0x33.	Báo cáo thông số thực đo 16 MiB; --eraseblock=0x4000 khớp với mtdinfo.

6.2. Kết luận

Lab 2 đạt các mục tiêu chính trong phạm vi kiểm thử đã ghi nhận: character device ARM hoạt động qua /dev/lab2; procfs/sysfs cung cấp được thống kê; BusyBox init nạp driver tự động trong QEMU; NAND giả lập trên HOST hỗ trợ erase, ghi image JFFS2, mount read-write và lưu dữ liệu qua remount. Header 85 19 được ghi nhận trong raw NAND dump, phù hợp định dạng JFFS2.

Cần phân biệt các bằng chứng có được với kiểm thử chưa thực hiện: ảnh không chứng minh persistence sau tắt HOST, độ bền NAND vật lý, độ ổn định race-concurrency, hay OTA hoàn chỉnh. Những nội dung này là hướng kiểm thử mở rộng, không phải kết quả đã đo.